

UNIVERSIDADE FEDERAL DE ALFENAS – UNIFAL-MG
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO
DISCIPLINA: TRABALHO DE CONCLUSÃO DE CURSO I

GUSTAVO BORIN NASCIMENTO

**AVALIAÇÃO DE RANGE PROOFS EM NOIR/ULTRAHONK PARA
VALIDAÇÃO PRIVADA DE ATRIBUTOS COMPROMETIDOS EM
CONTRATOS INTELIGENTES COMPATÍVEIS COM ETHEREUM**

Projeto de Trabalho de Conclusão de Curso apresentado ao Bacharelado em Ciência da Computação da Universidade Federal de Alfenas como requisito parcial para aprovação em Trabalho de Conclusão de Curso I.

Orientador: Prof. Iago Augusto de Carvalho

Resumo

Esta proposta de Trabalho de Conclusão de Curso investiga o uso de *Zero-Knowledge Range Proofs* para validação privada de atributos comprometidos em contratos inteligentes compatíveis com Ethereum. O problema central consiste em permitir que um usuário prove que um valor privado v pertence a um intervalo público $[a, b]$, sem revelar v à rede, e, simultaneamente, que esse valor está vinculado a um compromisso público C . A relação avaliada não pretende comprovar, por si só, a origem institucional do atributo, como ocorreria em uma credencial assinada por emissor confiável; seu escopo é medir a viabilidade do predicado de intervalo associado a um compromisso verificável. A pesquisa propõe desenvolver e avaliar circuitos de *Range Proof* no ecossistema Noir, utilizando ACIR como representação intermediária e Barretenberg/UltraHonk como backend de prova. O estudo será conduzido como uma avaliação aplicada de desempenho, considerando dois eixos principais: o ambiente de execução do provador, com comparação entre execução nativa e WebAssembly (WASM32), e o custo criptográfico de restrições de intervalo com diferentes larguras numéricas. Para evitar comparações semanticamente frágeis, a proposta distingue uma linha de base **Field** irrestrita, usada apenas como diagnóstico, de variantes **Field** com restrição explícita de tamanho de bits, usadas como linha de base semanticamente controlada para comparação com **u8**, **u32** e **u64**. A avaliação considerará métricas empíricas, especialmente tempo de geração da prova, e métricas estruturais, como tamanho da prova, complexidade do circuito, contagem de gates do backend, custo de verificação na EVM e custos de publicação de dados via *calldata*. Projeções com *blob gas* serão tratadas exclusivamente como cenários de loteamento ou rollup, não como substituto direto de *calldata* em um verificador autônomo na L1. Para lidar com variabilidade de latência em ambiente web, a metodologia prevê estudo piloto, execuções de aquecimento, randomização em blocos, medições repetidas por condição, intervalos de confiança por *bootstrap* e aplicação de testes estatísticos não paramétricos, especialmente o teste de Brunner–Munzel com correção de Holm–Bonferroni para múltiplas comparações. Também serão realizados testes negativos e checagens de sub-restrição antes da coleta de desempenho. Como contribuição esperada, o trabalho busca fornecer um caminho reproduzível para avaliar a viabilidade de *Range Proofs* em Noir sob restrições práticas de clientes WASM32 e contratos inteligentes EVM.

Palavras-chave: Provas de Conhecimento Zero; Range Proofs; Noir; UltraHonk; Ethereum; contratos inteligentes; WebAssembly; privacidade computacional.

Abstract

This undergraduate thesis project investigates the use of Zero-Knowledge Range Proofs to validate committed private attributes in Ethereum-compatible smart contracts. The central problem is to allow a user to prove that a private value v belongs to a public interval $[a, b]$ without revealing v to the network, while also proving that the value is bound to a public commitment C . The evaluated relation does not, by itself, prove the institutional origin of the attribute, as a signed credential would; rather, it measures the feasibility of an interval predicate bound to a verifiable commitment. The study proposes the development and evaluation of Range Proof circuits in the Noir ecosystem, using ACIR as the intermediate representation and Barretenberg/UltraHonk as the proving backend. The research is designed as an applied performance evaluation along two main axes: the prover execution environment, comparing native execution and WebAssembly (WASM32), and the cryptographic cost of range restrictions under different numeric widths. To avoid semantically weak comparisons, the proposal distinguishes an unrestricted `Field` baseline, used only as a diagnostic lower bound, from explicitly bit-constrained `Field` variants, used as semantically controlled baselines against `u8`, `u32`, and `u64`. The evaluation considers empirical metrics, especially proving time, and structural metrics such as proof size, circuit complexity, backend gate count, EVM verification gas, and calldata publication costs. Blob gas analysis is treated only as a projection for batching or rollup-oriented architectures, not as a direct replacement for calldata in a standalone L1 verifier. To address latency variability in browser-based proving, the methodology adopts a pilot study, burn-in runs, block randomization, repeated measurements per condition, bootstrap confidence intervals, and non-parametric statistical testing with the Brunner–Munzel test, with Holm–Bonferroni correction for multiple comparisons. Negative tests and under-constrained circuit checks will be performed before performance measurement. The expected contribution is a reproducible evaluation framework for assessing whether Noir-based Range Proofs can support privacy-preserving attribute validation under the architectural constraints of WASM32 clients and EVM smart contracts.

Keywords: Zero-Knowledge Proofs; Range Proofs; Noir; UltraHonk; Ethereum; smart contracts; WebAssembly; privacy-preserving computation.

Sumário

1	Introdução	5
1.1	Contextualização e motivação	5
1.2	Problema de pesquisa	6
1.3	Objetivos	6
1.4	Questão de pesquisa e hipóteses	7
1.5	Escopo e delimitações	8
1.6	Organização do texto	8
2	Revisão Bibliográfica	9
2.1	Fundamentos de Zero-Knowledge Proofs	9
2.2	Contratos Inteligentes e Ethereum Virtual Machine	11
2.3	Range Proofs: fundamentos e aplicações	12
2.4	A Linguagem Noir e o ecossistema Aztec	12
2.5	WebAssembly em aplicações criptográficas	13
2.6	Trabalhos relacionados e lacuna de pesquisa	14
3	Metodologia de Pesquisa	14
3.1	Classificação da pesquisa	14
3.2	Relação criptográfica avaliada	14
3.3	Fixação tecnológica e reprodutibilidade	15
3.4	Delineamento experimental	16
3.5	Protocolo de execução	16
3.6	Hipóteses estatísticas e tamanho de efeito	17
3.7	Métricas de avaliação	18
3.8	Validação de correção lógica	19
4	Resultados Esperados	20
4.1	Correção funcional	20
4.2	Comportamento do tempo de geração da prova	20
4.3	Custo de largura de bits e lookups	20
4.4	Verificação EVM e custos de dados	21

4.5	Artefatos esperados	21
4.6	Contribuição esperada	21
5	Cronograma Proposto para o TCC II	22
6	Considerações Finais da Proposta	22
A	Mapeamento técnico preliminar dos circuitos em Noir	23
A.1	Circuito conceitual de intervalo puro	23
A.2	Circuito conceitual com compromisso público	23
A.3	Variante Field com restrição explícita de bits	24
A.4	Testes negativos mínimos	24

1 Introdução

1.1 Contextualização e motivação

Blockchains públicas oferecem um ambiente transparente para execução de contratos inteligentes. Essa transparência é importante para auditabilidade, consenso e verificação independente, mas também cria um custo de privacidade: regras contratuais frequentemente exigem que usuários revelem exatamente os dados necessários para comprovar uma condição. Um contrato que verifica solvência, idade, saldo de tokens ou enquadramento em uma regra de conformidade pode precisar saber apenas se determinado valor satisfaz uma condição, mas o modelo padrão de execução em redes públicas expõe o valor à rede.

As Provas de Conhecimento Zero (*Zero-Knowledge Proofs*, ZKPs) oferecem uma alternativa a esse problema ao permitir que um provador convença um verificador de que determinada afirmação é verdadeira sem revelar o *witness* subjacente. Neste trabalho, a afirmação central é uma relação de intervalo: um valor privado v pertence a um intervalo público $[a, b]$. Uma *Range Proof* permite comprovar uma propriedade sobre um valor oculto, revelando apenas que o valor satisfaz limites públicos previamente definidos.

Entretanto, para que essa comprovação tenha significado criptográfico além de “existe algum valor no intervalo”, é necessário vincular v a algum artefato público verificável. Por isso, esta proposta passa a tratar explicitamente o valor privado como atributo comprometido: além de provar $a \leq v \leq b$, o circuito deve provar que v é consistente com um compromisso público C . Esse compromisso pode ser implementado por um hash ZK-friendly, por um compromisso criptográfico ou, em trabalhos futuros, por uma credencial assinada por emissor confiável. No escopo deste TCC, o foco permanece na avaliação do predicado de intervalo e de seu vínculo mínimo com um compromisso público, sem implementar uma infraestrutura completa de identidade, KYC ou emissão de credenciais.

A adoção prática de *Range Proofs* depende tanto de correção criptográfica quanto de viabilidade de engenharia. Uma prova que preserva privacidade, mas demora demais para ser gerada no dispositivo do usuário, produz artefatos grandes ou exige custo excessivo de verificação na EVM, pode ser inadequada para fluxos reais de contratos inteligentes. Esse problema é especialmente relevante quando a prova é gerada no lado do cliente, em ambiente de navegador, por meio de WebAssembly (WASM32), pois esse ambiente possui limites de memória, características de compilação, comportamento de coleta de lixo e modelo de concorrência diferentes da execução nativa.

Noir surge nesse contexto como uma linguagem de alto nível para escrita de circuitos ZK, com compilação para ACIR, uma representação intermediária consumida por backends de prova como Barretenberg. No ecossistema Noir, comparações e restrições inteiras podem ser expressas com construções próximas às de linguagens de programação convencionais,

enquanto o backend traduz essas operações para constraints aritméticas, tabelas de lookup e artefatos de prova. Essa abstração melhora a ergonomia do desenvolvedor, mas não elimina o custo computacional de comparações, restrições de intervalo e verificações on-chain.

1.2 Problema de pesquisa

O problema de pesquisa deste TCC é avaliar a viabilidade prática de provar, em contratos inteligentes compatíveis com Ethereum, que um atributo privado comprometido pertence a um intervalo público, usando *Zero-Knowledge Range Proofs* escritas em Noir e provadas com Barretenberg/UltraHonk.

A relação central avaliada pode ser expressa como:

$$\mathcal{R}_{\text{attr}} = \{(a, b, C, \text{id}; v, r) : a \leq v \leq b \wedge C = \text{Commit}(v, r, \text{id})\}, \quad (1)$$

em que a e b são limites públicos, C é um compromisso público, id é um identificador público ou domínio de aplicação, v é o atributo privado e r é a aleatoriedade privada usada no compromisso. Essa formulação evita a interpretação mais fraca segundo a qual o provador apenas conhece algum valor arbitrário no intervalo. Ao mesmo tempo, o TCC não afirma comprovar a autenticidade institucional de v sem uma credencial externa assinada; essa camada fica explicitamente fora do escopo.

A dificuldade científica e técnica está em determinar o custo dessa garantia de privacidade quando o circuito é compilado, provado e verificado por uma cadeia de ferramentas real. Esse problema possui duas dimensões principais. A primeira é arquitetural: o provador pode executar como binário nativo ou dentro de um navegador via WASM32, e esses ambientes não apresentam o mesmo comportamento de desempenho. A segunda é representacional e criptográfica: o uso de tipos inteiros restritos, como `u8`, `u32` e `u64`, pode introduzir overhead associado a restrições de largura de bits e mecanismos de lookup quando comparado a variantes baseadas em `Field` com domínio controlado.

1.3 Objetivos

O objetivo geral deste trabalho é desenvolver, implantar e avaliar circuitos de *Zero-Knowledge Range Proof* em Noir para validação privada de atributos comprometidos em contratos inteligentes compatíveis com Ethereum.

Os objetivos específicos são:

1. Implementar circuitos Noir que provem a relação $a \leq v \leq b$ sem revelar o valor privado v .

2. Incluir uma variante vinculada a compromisso público que garanta, para o mesmo v do predicado de intervalo, a relação

$$C = \text{Commit}(v, r, \text{id}).$$

3. Compilar os circuitos por meio da cadeia Noir/Barretenberg, utilizando ACIR e UltraHonk como pipeline de prova.
4. Gerar e implantar um verificador Solidity compatível com EVM, considerando restrições de tamanho de bytecode, como o EIP-170.
5. Comparar ambientes de prova nativo e WASM32 por meio de medições repetidas de latência.
6. Avaliar o efeito de diferentes larguras numéricas, distinguindo linha de base `Field` irrestrita, `Field` com restrição explícita de bits e variantes `u8`, `u32` e `u64`.
7. Medir artefatos estruturais, incluindo tamanho da prova, opcodes ACIR, gates do backend, *execution gas* e custos de publicação de dados.
8. Validar a correção lógica dos circuitos por meio de testes negativos com entradas inválidas e checagens contra sub-restrição.

1.4 Questão de pesquisa e hipóteses

A questão de pesquisa que orienta este TCC é:

Quão viável é uma *Range Proof* Noir/UltraHonk para validação privada de atributos comprometidos em contratos inteligentes compatíveis com Ethereum, quando avaliada em ambientes de prova nativo e WASM32 e sob diferentes larguras numéricas?

Duas hipóteses empíricas estruturam a avaliação de desempenho. A primeira refere-se ao ambiente de execução: espera-se que a geração de provas em ambiente nativo apresente menor latência que a execução em WASM32, em razão de restrições associadas ao navegador, ao gerenciamento de memória, à compilação dinâmica e ao modelo de concorrência. A segunda hipótese refere-se ao custo das restrições de intervalo: espera-se que variantes baseadas em tipos inteiros restritos apresentem overhead mensurável em relação a uma linha de base `Field` semanticamente controlada, uma vez que restrições de largura de bits e comparações exigem trabalho adicional do sistema de prova.

Essas hipóteses serão aplicadas às métricas estocásticas, principalmente tempo de geração de prova. Métricas estruturais, como tamanho da prova, contagem de gates

e gas de verificação para um artefato fixo, serão comparadas diretamente como saídas do pipeline de compilação e verificação, com exceção dos custos de *calldata*, que serão calculados sobre os bytes concretos das provas coletadas.

1.5 Escopo e delimitações

Este trabalho concentra-se no predicado de validação de intervalo e exclui, deliberadamente, lógicas de negócio complexas. O circuito avaliado prova que um atributo privado pertence a um intervalo público e, na variante vinculada, que esse atributo corresponde a um compromisso público. Não será implementada uma aplicação financeira, de identidade, votação ou conformidade completa. Essa delimitação permite isolar o custo do predicado criptográfico, evitando que os resultados sejam confundidos por regras de negócio externas.

Também fica fora do escopo a emissão de credenciais por terceiros e a verificação de assinaturas de emissores confiáveis dentro do circuito. Assim, o trabalho não afirmará provar que o atributo foi emitido por uma instituição real. A afirmação tecnicamente sustentada será: dado um compromisso público C , o provador conhece um par privado (v, r) tal que v pertence ao intervalo público $[a, b]$ e C é consistente com (v, r) sob a função de compromisso especificada.

O trabalho limita-se à verificação compatível com EVM. A rede Sepolia será usada como referência pública de testnet, enquanto a medição de gas será conduzida preferencialmente em fork local para reduzir ruídos de RPC, mempool e propagação de transações. A análise considerará custos de execução e de publicação de dados em um contexto pós-Pectra, distinguindo custos via *calldata* de projeções associadas a cenários com blobs. A análise de *blob gas* será tratada como projeção para arquiteturas de loteamento ou rollups, e não como substituto direto do *calldata* em um verificador autônomo na L1.

A avaliação de prova em navegador será limitada à execução WASM32 em condições controladas. O estudo não pretende generalizar os resultados para todos os dispositivos, navegadores ou classes de hardware, mas sim explicitar a penalidade arquitetural do ambiente web no cenário experimental escolhido.

1.6 Organização do texto

Além desta introdução, o texto apresenta uma revisão bibliográfica sobre ZKPs, *Range Proofs*, contratos inteligentes, EVM, Noir, Barretenberg, UltraHonk, WASM e trabalhos relacionados. Em seguida, descreve a metodologia de pesquisa, incluindo variáveis, hipóteses, desenho experimental, métricas, estratégia de análise estatística e controles de reprodutibilidade. Posteriormente, são apresentados os resultados esperados, o cronograma proposto para o TCC II e as considerações finais desta proposta.

2 Revisão Bibliográfica

2.1 Fundamentos de Zero-Knowledge Proofs

O conceito de *zero-knowledge proofs* surge na teoria de complexidade de prova, a partir da tentativa de quantificar quanta informação além da veracidade de um enunciado é transmitida em um protocolo de prova. Goldwasser, Micali e Rackoff introduziram a noção de complexidade de conhecimento e definiram formalmente provas de conhecimento zero, mostrando que é possível convencer um verificador da veracidade de certas propriedades sem revelar o *witness* subjacente.

Formalmente, uma ZKP é um protocolo entre um provador P e um verificador V para uma linguagem L em NP, associada a uma relação R tal que $x \in L$ se, e somente se, existe um witness w com $R(x, w) = 1$. O protocolo deve satisfazer três propriedades fundamentais:

- **Completeness:** para todo $x \in L$ e witness válido w , um provador honesto consegue convencer um verificador honesto com probabilidade próxima de 1.
- **Soundness:** para $x \notin L$, nenhum provador malicioso sem acesso a um witness consegue convencer um verificador honesto, exceto com probabilidade negligenciável.
- **Conhecimento zero:** existe um simulador eficiente que, sem acesso ao witness, gera transcrições indistinguíveis das obtidas na interação real, garantindo que o verificador não aprende nada além da validade do enunciado.

No contexto deste TCC, os circuitos Noir codificam relações de intervalo e, na variante vinculada, uma relação de consistência com compromisso público. As garantias criptográficas de completeness, soundness computacional e conhecimento zero são tratadas como garantias do sistema de prova UltraHonk/Barretenberg apenas sob hipóteses explícitas: a relação deve estar corretamente especificada no circuito, não deve haver sub-restrição, as versões do backend devem estar fixadas, os parâmetros devem ser usados conforme a documentação e os testes negativos devem confirmar que entradas inválidas não produzem provas aceitas. Dessa forma, evita-se a afirmação excessivamente forte de que o backend, por si só, garante que o atributo modelado pelo pesquisador tem o significado pretendido.

Provas interativas, NIZKs e transformação de Fiat–Shamir

As ZKPs foram inicialmente formuladas como protocolos interativos, em que verificador e provador trocam múltiplas mensagens, com o verificador escolhendo desafios

aleatórios a cada rodada. Em ambientes distribuídos como blockchains, entretanto, é desejável eliminar essa interação, substituindo-a por uma prova única que o verificador possa checar off-line: as chamadas *Non-Interactive Zero-Knowledge Proofs* (NIZKs).

A transformação de Fiat–Shamir converte protocolos públicos de desafio-resposta em provas não interativas, substituindo os desafios aleatórios do verificador por chamadas a uma função hash modelada como oráculo aleatório. Dado um transcript parcial T , o desafio c pode ser definido como $c = H(T)$, eliminando a necessidade de um verificador ativo durante a prova. Essa técnica é usada tanto para derivar NIZKs quanto para construir esquemas de assinatura a partir de protocolos de identificação.

SNARKs e STARKs

Os *Succinct Non-interactive ARguments of Knowledge* (SNARKs) são NIZKs com provas pequenas e tempo de verificação reduzido em relação ao tamanho do witness, propriedades cruciais para aplicações em blockchains. O esquema Groth16, por exemplo, oferece provas de tamanho constante e verificação rápida, à custa de um *trusted setup* específico para cada circuito.

PLONK introduz uma família de SNARKs universais e atualizáveis, baseados em compromissos polinomiais e em um argumento de permutação sobre avaliações em uma base de Lagrange. Extensões PLONKish agregam *lookup tables*, *custom gates* e otimizações para recursão, permitindo expressar operações inteiras, comparações e *range checks* com menos gates do que em circuitos puramente aritméticos.

Os *Scalable Transparent Arguments of Knowledge* (STARKs) são argumentos transparentes, baseados sobretudo em funções hash, códigos de correção de erros e protocolos FRI. STARKs evitam *trusted setup* e oferecem boa escalabilidade para grandes volumes de computação, mas produzem provas maiores do que muitos SNARKs emparelhados, o que impacta custos de transmissão e armazenamento em blockchains.

UltraHonk, lookup tables e Sumcheck

No ecossistema Aztec/Noir, a biblioteca Barretenberg implementa uma família PLONKish otimizada, incluindo UltraPlonk e UltraHonk. UltraHonk combina ideias de PLONK com protocolos de Sumcheck sobre representações multilineares, visando reduzir pressões de memória e melhorar a escalabilidade do provador em circuitos grandes.

UltraHonk não deve ser descrito apenas como “PLONK com lookups”. Parte central da família Honk é substituir o polinômio quociente global de PLONK clássico por um protocolo de Sumcheck sobre o hiper cubo booleano, o que reduz a necessidade de materializar polinômios de alto grau associados a circuitos grandes na memória. Em vez

de construir um único polinômio quociente cujo grau cresce com o tamanho do circuito, o provador Honk usa Sumcheck para provar que a soma de avaliações de polinômios multilineares satisfaz uma relação esperada.

No contexto deste TCC, essa característica é crítica: ao medir *proving time* em ambientes nativo e WASM32, o experimento captura como um backend baseado em Sumcheck se comporta em arquiteturas com diferentes padrões de cache, latência de memória e limites de heap.

2.2 Contratos Inteligentes e Ethereum Virtual Machine

Nick Szabo introduziu o termo *smart contracts* na década de 1990 para descrever acordos autoexecutáveis cujos termos são implementados diretamente em hardware ou software, em vez de depender exclusivamente de instituições legais. O Ethereum White Paper, posteriormente, propôs uma plataforma de contratos inteligentes sobre uma blockchain programável, definindo um modelo de máquina de estados global em que cada transação é uma transição determinística aplicada ao estado corrente.

A Ethereum Virtual Machine (EVM) é uma máquina de pilha de 256 bits com conjunto de instruções próprio, projetada para ser determinística e isolada. Cada instrução consome uma quantidade de gas, unidade abstrata de custo computacional e de armazenamento usada para limitar o consumo de recursos em uma transação e prevenir ataques de negação de serviço.

Para verificadores ZK on-chain, gas é uma métrica central de custo. Operações como multiplicações em curvas elípticas, leituras de *calldata* e manipulação de memória possuem custos distintos. Neste TCC, o contrato verificador UltraHonk gerado por Barretenberg será avaliado quanto ao *execution gas*, ao custo de *calldata* das provas concretas e a projeções separadas para cenários com blobs.

EIP-170, EIP-4844 e EIP-7623

O EIP-170 introduz um limite máximo de 24.576 bytes para o tamanho do bytecode de contratos. Caso o bytecode retornado na criação de um contrato exceda esse limite, a criação falha. Verificadores SNARK gerados automaticamente podem produzir bytecode próximo ou superior a esse limite, exigindo estratégias como otimização de compilação ou arquitetura de verificador particionado.

O EIP-4844 introduz transações com blobs, anexando grandes blocos de dados a um tipo específico de transação e criando um mercado separado denominado *blob gas*. Entretanto, o conteúdo dos blobs não é acessível diretamente pela execução EVM; contratos só podem observar compromissos ou hashes versionados. Portanto, para um contrato

verificador standalone em L1, a prova precisa continuar disponível em um formato que a EVM consiga ler, normalmente *calldata*. Assim, qualquer análise de blobs neste TCC será tratada apenas como projeção para arquiteturas de loteamento ou rollup.

O EIP-7623 altera o cálculo de custo de transações com grande volume de *calldata*, introduzindo um piso baseado em tokens de *calldata*. Como provas ZK são frequentemente transmitidas como *calldata*, o custo de dados deve ser calculado sobre os bytes concretos de cada prova, diferenciando bytes zero e não zero e aplicando a fórmula apropriada ao ambiente EVM considerado.

2.3 Range Proofs: fundamentos e aplicações

Uma *Range Proof* é uma prova de conhecimento zero que demonstra que um valor secreto v pertence a um intervalo, sem revelar v . A formulação típica envolve um compromisso criptográfico a v e uma prova de que v satisfaz uma faixa válida. Em termos de relações NP, o enunciado público inclui o intervalo e o compromisso, enquanto o witness inclui v e a aleatoriedade do compromisso. A prova convence o verificador de que existe um witness consistente com a relação “ v está no intervalo e C é compromisso de v ”.

Essa tarefa é criptograficamente não trivial porque o compromisso precisa satisfazer propriedades de ocultação e vinculação. Compromissos de Pedersen, por exemplo, são perfeitamente ocultantes e computacionalmente vinculantes sob hipótese de logaritmo discreto, desde que os geradores sejam escolhidos de forma adequada e que o logaritmo discreto relativo entre eles seja desconhecido.

Bulletproofs introduzem *Range Proofs* com tamanho logarítmico em relação ao número de bits do intervalo, sem necessidade de trusted setup. A técnica se baseia em *inner-product arguments* recursivos. Range Proofs baseadas em SNARKs, por sua vez, codificam o predicado de intervalo como circuito aritmético e depois usam um SNARK de propósito geral para comprovar a satisfatibilidade desse circuito.

Em sistemas SNARK/PLONKish, espera-se que o tamanho da prova cresça muito menos que o circuito e possa permanecer aproximadamente estável para variações pequenas do predicado. Entretanto, a invariância absoluta não deve ser pressuposta: o tamanho efetivo da prova depende do backend, do número de entradas públicas, do formato do objeto de prova, dos parâmetros do protocolo e da versão das ferramentas. Por isso, este TCC medirá empiricamente o tamanho da prova para cada variante.

2.4 A Linguagem Noir e o ecossistema Aztec

Noir é uma linguagem de domínio específico para construção de circuitos de conhecimento zero, desenvolvida pela Aztec Labs e disponibilizada como projeto open-source. A

linguagem é projetada para abstrair parte da complexidade criptográfica, permitindo que desenvolvedores escrevam lógica de prova em estilo de alto nível, enquanto o compilador gera uma representação aritmética adequada a diferentes sistemas de prova.

A documentação de Noir descreve um pipeline em que o código Noir é compilado em ACIR (*Abstract Circuit Intermediate Representation*), uma representação intermediária backend-agnóstica. Em seguida, ACIR é consumido por backends como Barretenberg, que produzem provas, chaves de verificação e, quando aplicável, contratos verificadores Solidity.

Noir oferece tipos `Field` e tipos inteiros de tamanho explícito. O tipo `Field` corresponde ao campo nativo do backend e tende a ser eficiente, mas operações de aritmética de campo envolvem wraparound modular e não equivalem automaticamente à semântica usual de inteiros limitados. Tipos como `u8`, `u32` e `u64`, por outro lado, impõem restrições de faixa e checagens de overflow, o que torna sua semântica mais próxima de inteiros convencionais, mas pode aumentar o custo do circuito. Essa distinção fundamenta a decisão metodológica de separar `Field_unconstrained` de `Field` com restrição explícita de bits.

2.5 WebAssembly em aplicações criptográficas

WebAssembly é um formato de código de baixo nível, seguro e portátil, padronizado como alvo de compilação para linguagens como C, C++ e Rust. O objetivo é permitir que programas de alto desempenho sejam executados em navegadores e outros ambientes com desempenho próximo ao nativo, dentro de um sandbox que garante isolamento de memória.

Motores como V8 adotam pipelines de compilação com estágios diferentes, combinando compilação rápida de início com recompilação otimizada de funções quentes. Esse processo pode introduzir efeitos de *cold-start* e variações ao longo da execução. Em workloads criptográficos, como geração de provas ZK, o provador precisa executar operações de alta complexidade, incluindo FFTs, multiplicações em campos finitos, operações em curvas elípticas e manipulação de grandes vetores de dados. Essas operações exercem pressão sobre heap, largura de banda de memória e sincronização.

Para suportar multithreading em navegador, o ecossistema web combina Web Workers, `SharedArrayBuffer` e operações atômicas. Esse modelo difere do uso de threads nativas do sistema operacional. Portanto, ainda que se tente igualar o número de workers e threads, a comparação entre nativo e WASM32 mede diferenças reais de runtime, sandbox, sincronização e política de agendamento.

2.6 Trabalhos relacionados e lacuna de pesquisa

A literatura recente contém esforços significativos para mensurar o desempenho de sistemas ZK de forma rigorosa. Trabalhos de benchmarking de ZKPs propõem métricas como *proving time*, *verification time*, tamanho da prova, uso de memória e largura de banda. Ferramentas como zk-Bench e estudos sobre PLONK, Groth16, Halo2 e outros frameworks oferecem pontos de comparação metodológica.

Entretanto, não foram localizados trabalhos com exatamente o perfil deste TCC: benchmarking sistemático de provedores Noir/UltraHonk em execução nativa versus WASM32, com controle explícito de bit-width e instrumentação de gas em contratos verificadores EVM sob restrições como EIP-170, EIP-4844 e EIP-7623. O presente trabalho busca preencher essa lacuna ao desenhar um experimento que isola variáveis de representação numérica, ambiente de execução e custo on-chain.

3 Metodologia de Pesquisa

A presente pesquisa propõe o desenvolvimento, a implementação e a avaliação empírica de desempenho de circuitos criptográficos do tipo *Range Proof* construídos no ecossistema Noir. O objetivo central é avaliar empiricamente o custo computacional e a degradação de desempenho desses circuitos, quantificando o impacto arquitetural do ambiente de execução e o peso criptográfico da imposição de diferentes larguras de bits sobre tempo de geração da prova, tamanho do artefato e custo de verificação na EVM.

3.1 Classificação da pesquisa

A pesquisa caracteriza-se como aplicada, experimental e quantitativa, com avaliação de desempenho de artefatos computacionais. O objeto de estudo não é apenas a formulação abstrata de uma Range Proof, mas sua implementação em uma cadeia de ferramentas real: Noir, ACIR, Barretenberg/UltraHonk, WASM32 e EVM.

3.2 Relação criptográfica avaliada

Serão avaliadas duas camadas de relação:

- a) **Relação de intervalo pura:** $\mathcal{R}_{\text{range}} = \{(a, b); v : a \leq v \leq b\}$. Essa variante mede o custo mínimo do predicado de intervalo, mas será descrita explicitamente como relação fraca, pois não vincula v a um atributo verificável.

- b) **Relação de atributo comprometido:** $\mathcal{R}_{\text{attr}} = \{(a, b, C, \text{id}); v, r : a \leq v \leq b \wedge C = \text{Commit}(v, r, \text{id})\}$. Essa será a relação principal para sustentar a expressão “atributo comprometido”.

A função **Commit** será implementada com uma primitiva compatível com circuitos Noir e disponível na stack fixada. A escolha operacional poderá ser um hash ZK-friendly, como Poseidon/Poseidon2, ou um compromisso equivalente suportado de forma estável no ecossistema. O relatório final registrará a primitiva escolhida, versão da biblioteca, número de constraints adicionais e impacto sobre proving time.

3.3 Fixação tecnológica e reprodutibilidade

Para garantir reprodutibilidade, não serão usados critérios abertos como “versão compatível” ou “versão maior ou igual”. Antes da coleta final, o repositório do TCC deverá conter um arquivo de ambiente com as versões exatas, hashes e comandos de reprodução. A Tabela 1 define o protocolo de fixação.

Tabela 1: Matriz de stack pinning e metadados obrigatórios.

Componente	Valor a fixar	Evidência no repositório
Noir CLI / nargo	Versão exata, sem operador $\geq$. Versão candidata inicial: linha estável/beta indicada pela documentação oficial no início do TCC II.	Saída de nargo -version ; lockfile ou script de instalação.
Barretenberg CLI / bb	Versão exata compatível com o nargo fixado.	Saída de bb -version ; hash do binário ou release.
@noir-lang/noir_js	Versão exata no package-lock.json .	package-lock.json versionado.
@aztec/bb.js	Versão exata no package-lock.json .	package-lock.json versionado.
Node.js / npm	Versões exatas.	Saída de node -version e npm -version .
Solidity / solc	Versão exata do compilador.	Saída de solc -version ; configuração do Foundry.
Foundry / anvil	Tag, commit ou versão exata.	Saída de forge -version e anvil -version .
Chrome / V8	Versão completa do navegador e V8.	chrome://version exportado nos logs.
Sistema operacional	Distribuição, versão, kernel e arquitetura.	Saída de uname -a , lsb_release -a ou equivalente.
Hardware	CPU, núcleos, RAM, governador de frequência e política térmica.	Saída de lscpu , free -h e configuração de energia.
Artefatos gerados	ACIR, witness, proof, verification key e verifier Solidity.	Hash SHA-256 de cada artefato.

O protocolo experimental só será considerado válido se as coletas nativa e WASM32

utilizarem a mesma versão de circuito, mesma witness policy, mesmas entradas públicas e o mesmo conjunto de artefatos compilados, exceto quando a diferença fizer parte da variável experimental.

3.4 Delineamento experimental

O experimento possui duas variáveis independentes principais:

1. **Ambiente de execução do provador:** execução nativa via CLI Barretenberg versus execução em navegador via WASM32.
2. **Representação numérica:** `Field_unconstrained`, `Field_asserted_N`, `u8`, `u32` e `u64`.

A variante `Field_unconstrained` será usada apenas como linha de base diagnóstica inferior, pois pode provar uma relação mais fraca. Para comparação semanticamente controlada, cada largura $N \in \{8, 32, 64\}$ terá uma variante `Field_asserted_N`, em que v , a e b terão domínio restringido explicitamente por `assert_max_bit_size<N>` ou mecanismo equivalente. Assim, a comparação `Field_asserted_N` versus `uN` mede diferenças de implementação e custo de tipos, não uma mudança silenciosa de semântica.

3.5 Protocolo de execução

A coleta empírica da latência seguirá três fases.

Fase 0: protótipo mínimo de viabilidade

Antes da implementação completa, será executado um protótipo mínimo contendo: circuito simples, geração de proof, geração de verification key, geração de verifier Solidity, medição do tamanho do bytecode e tentativa de deploy em ambiente local. Essa fase será realizada no início do TCC II, porque o limite do EIP-170 pode afetar a viabilidade do verificador UltraHonk. O resultado produzirá um plano A/B/C:

- **Plano A:** verifier gerado cabe no limite com otimização padrão.
- **Plano B:** verifier cabe apenas com configuração agressiva de otimização.
- **Plano C:** é necessária arquitetura de split-verifier ou redução de escopo do circuito.

Fase 1: estudo piloto

Será conduzido um estudo piloto com, no mínimo, 20 medições por condição experimental para estimar variância, caudas de distribuição e estabilidade do burn-in. O objetivo do piloto é calibrar K e N , não testar hipóteses finais. A configuração inicial será $K = 15$ execuções descartadas e $N = 100$ execuções registradas por condição, mas esses valores poderão ser ajustados caso o piloto revele variância excessiva, falhas frequentes ou instabilidade de tier-up no V8.

Fase 2: coleta final randomizada

A coleta final será organizada em blocos. Cada bloco conterá uma execução de cada variante de circuito, com ordem pseudoaleatória definida por seed fixa e registrada nos logs. O mesmo conjunto de witnesses válidos será usado de forma pareada quando isso não alterar a relação do circuito. A estrutura mínima de log por execução incluirá: identificador do bloco, variante, ambiente, seed, timestamp, hash do circuito, hash da proof, proving time, status de sucesso/falha, uso aproximado de memória quando disponível e metadados do runtime.

No ambiente WASM32, a execução seguirá controles adicionais:

- página HTML/JS mínima dedicada ao benchmark;
- servidor local com cabeçalhos COOP/COEP habilitados;
- verificação programática de `crossOriginIsolated === true`;
- registro do número real de workers inicializados;
- execução em janela ativa, em perfil limpo do navegador, sem extensões e sem abas extras;
- separação entre tempo de carregamento do WASM, geração do witness e geração da prova, sempre que tecnicamente possível.

3.6 Hipóteses estatísticas e tamanho de efeito

A testagem estatística será restrita às variáveis empíricas que apresentam flutuação estocástica, principalmente o tempo de geração da prova. Variáveis estruturais serão analisadas por comparação direta, exceto quando dependerem de bytes concretos das provas coletadas, como *calldata gas*.

Eixo A: impacto arquitetural do ambiente de execução

- H_0 : não há diferença estocástica relevante no tempo de geração da prova entre execução nativa e execução via WASM32.
- H_1 : existe diferença estocástica entre os ambientes, com expectativa de latência superior no WASM32.

Além do p-valor, será reportado tamanho de efeito. Uma diferença será considerada praticamente relevante quando a razão entre medianas for igual ou superior a 1,20, ou quando a estatística de dominância estocástica indicar efeito no mínimo moderado, por exemplo $P(X_{\text{nativo}} < X_{\text{WASM}}) \geq 0,65$.

Eixo B: custo empírico da restrição de intervalo

- H_0 : a imposição de tipos inteiros restritos não gera diferença estocástica relevante no tempo de geração da prova em comparação às variantes `Field` semanticamente controladas.
- H_1 : a restrição de largura de bits incrementa a latência de geração da prova, refletindo overhead de range checks, decomposição ou lookup tables.

As comparações planejadas serão feitas em três pares principais:

- `Field_asserted_8` versus `u8`;
- `Field_asserted_32` versus `u32`;
- `Field_asserted_64` versus `u64`.

Também serão realizadas comparações diagnósticas com `Field_unconstrained`. Será aplicada correção de Holm–Bonferroni para múltiplas comparações. A análise reportará mediana, intervalo interquartilico, ECDF, intervalos de confiança por bootstrap e resultado do teste de Brunner–Munzel.

3.7 Métricas de avaliação

As métricas serão divididas em empíricas e estruturais.

Métricas empíricas

1. **Tempo de geração da prova:** mensurado em milissegundos, separando, quando possível, carregamento, witness generation e proving.

2. **Taxa de falha:** proporção de execuções que não geram prova válida. Se uma condição apresentar falha superior a 15%, a análise de latência dessa condição será suspensa e a taxa de falha será reportada como resultado principal.

Métricas estruturais

1. **Tamanho da prova:** medido em bytes para cada proof concreta.
2. **Complexidade do circuito:** medida por opcodes ACIR, gates do backend e/ou métricas disponibilizadas pelo Barretenberg.
3. **Tamanho do verificador Solidity:** bytecode de criação e bytecode runtime, comparado ao limite do EIP-170.
4. **Execution gas:** gas consumido pela execução do verificador para proof e inputs fixos.
5. **Calldata gas bruto:** calculado a partir dos bytes concretos da proof e entradas públicas, diferenciando bytes zero e não zero.
6. **Piso pós-EIP-7623:** calculado conforme a fórmula aplicável de tokens de calldata.
7. **Projeção de blob gas:** estimativa separada e explicitamente hipotética para arquiteturas de rollup ou batching.

3.8 Validação de correção lógica

Antes de qualquer benchmark, serão executados testes negativos. Exemplos incluem:

- $v < a$ deve falhar;
- $v > b$ deve falhar;
- $a > b$ deve falhar;
- proof gerada com compromisso C incompatível deve falhar;
- salt incorreto ou identificador incorreto deve falhar;
- witness válido para uma variante não deve ser aceito por verifier de outra variante, salvo quando isso for explicitamente esperado.

Além disso, o compilador não deverá ser executado com flags que desabilitem checagens de sub-restrição em artefatos usados para coleta final. Caso alguma flag experimental seja necessária, ela será registrada e justificada.

4 Resultados Esperados

4.1 Correção funcional

Espera-se obter circuitos Noir funcionais capazes de provar o predicado $a \leq v \leq b$, mantendo v privado e expondo apenas os limites públicos necessários ao verificador. Na variante principal, espera-se também provar que v está vinculado ao compromisso público C , evitando que o provador escolha arbitrariamente qualquer valor no intervalo sem relação com o artefato público verificado.

Entradas válidas devem gerar provas aceitas, enquanto entradas inválidas devem falhar durante a geração ou verificação da prova. Assim, os testes negativos devem confirmar que o circuito não está sub-restrito e que alegações falsas sobre atributos comprometidos são rejeitadas.

4.2 Comportamento do tempo de geração da prova

Espera-se que a geração de provas em ambiente nativo seja mais rápida e estável que a geração em WASM32. A execução nativa de Barretenberg tem acesso direto ao modelo de memória do sistema operacional e evita restrições específicas do navegador. Já a execução WASM32 depende do motor do navegador, de limites de heap, de estágios de compilação, de disponibilidade de workers e de políticas de sandbox. Por isso, espera-se identificar uma penalidade de latência mensurável no ambiente web.

Também se espera que medições em WASM32 apresentem maior dispersão que medições nativas. Mesmo após aquecimento, a execução em navegador pode apresentar caudas longas causadas por agendamento do runtime, pressão de memória, coleta de lixo e efeitos de compilação. Por esse motivo, os resultados deverão ser descritos por gráficos de violino, funções de distribuição acumulada empírica, medianas, intervalos interquartílicos, intervalos de confiança e testes de dominância estocástica, em vez de médias isoladas.

4.3 Custo de largura de bits e lookups

Espera-se que `Field_unconstrained` apresente menor custo, mas essa variante será interpretada apenas como limite inferior diagnóstico, não como circuito semanticamente equivalente. As variantes `Field_asserted_N` fornecerão linhas de base mais justas para comparação com `u8`, `u32` e `u64`.

As variantes inteiras devem introduzir trabalho adicional de prova porque comparações inteiras e tipos numéricos limitados exigem restrições de largura de bits, decomposição ou mecanismos orientados a lookup no backend UltraHonk. Não se espera necessariamente

crescimento perfeitamente linear entre as larguras de bits, pois otimizações do backend podem decompor inteiros em limbs menores ou reutilizar estruturas de lookup. Ainda assim, espera-se diferença mensurável em relação às linhas de base.

4.4 Verificação EVM e custos de dados

Espera-se que o gas de execução da EVM para um verificador e proof fixos se comporte de forma determinística. Entretanto, o custo total de dados não será tratado como constante universal, porque o *calldata gas* depende dos bytes concretos da proof, especialmente da proporção entre bytes zero e não zero. Por isso, o custo de *calldata* será calculado para cada proof coletada ou para amostra representativa, e não apenas para um exemplo isolado.

Custos orientados a blobs deverão funcionar como projeção comparativa para arquiteturas de rollup ou batching, e não como modelo direto de custo para o verificador standalone. Essa distinção deverá esclarecer quais partes do sistema são diretamente implantáveis como contrato inteligente e quais dependeriam de infraestrutura adicional.

4.5 Artefatos esperados

Ao final da execução do TCC II, espera-se produzir:

- variantes de circuitos Noir;
- scripts de compilação e geração de witness;
- proofs, verification keys e verificadores Solidity;
- scripts de deploy ou simulação em anvil/Sepolia;
- logs brutos de benchmark;
- arquivos de metadados do ambiente;
- tabelas estatísticas e gráficos;
- relatório final com interpretação dos resultados e limitações.

4.6 Contribuição esperada

A contribuição esperada é técnica e metodológica. Do ponto de vista técnico, o trabalho deverá fornecer um caminho de implementação para validação privada de atributos comprometidos por intervalo em contratos inteligentes, usando Noir e UltraHonk.

Do ponto de vista metodológico, deverá oferecer um desenho de benchmark que separa o comportamento estocástico do provador de métricas estruturais de verificação e artefatos. Essa separação deve tornar a análise final mais confiável que uma única métrica agregada de desempenho.

5 Cronograma Proposto para o TCC II

O cronograma abaixo organiza a execução do TCC II em seis meses de trabalho. A distribuição poderá ser ajustada conforme o calendário oficial da disciplina, mas a ordem das atividades deve ser preservada para manter dependência lógica entre implementação, coleta de dados, análise e redação final.

Tabela 2: Cronograma revisado com antecipação de riscos técnicos.

Período	Atividades
Mês 1	Revisão final do escopo; fixação de versões; criação do repositório; protótipo mínimo de viabilidade; circuito mínimo compila; geração inicial do verifier Solidity; medição preliminar de bytecode sob EIP-170.
Mês 2	Implementação das variantes de circuito <code>Field</code> e inteiras; implementação da variante com compromisso; testes negativos.
Mês 3	Geração de proofs, verification keys e verificadores Solidity; validação nativa mínima; preparação de anvil/fork local; definição do plano A/B/C para deploy do verifier.
Mês 4	Implementação da rotina de benchmark nativo e WASM32; servidor local com COOP/COEP; validação de <code>crossOriginIsolated</code> ; estudo piloto; calibração de K , N e seed de randomização.
Mês 5	Coleta final; organização dos logs; cálculo de métricas estruturais; cálculo de gas, calldata e piso EIP-7623; análise estatística; geração de gráficos e tabelas.
Mês 6	Redação dos resultados; discussão; análise de limitações; conclusão; normalização do texto; revisão final; preparação para entrega ou defesa.

6 Considerações Finais da Proposta

Este texto consolida a etapa de TCC I como projeto de pesquisa. Sua função não é apresentar resultados empíricos já coletados, mas estabelecer com clareza o problema, o recorte, a fundamentação teórica, os objetivos, a metodologia e os resultados esperados para execução no TCC II.

A proposta delimita um problema tecnicamente específico e mensurável: avaliar a viabilidade de *Range Proofs* em Noir/UltraHonk para validação privada de atributos comprometidos em contratos inteligentes compatíveis com Ethereum. A escolha por comparar execução nativa e WASM32, bem como variantes `Field`, `u8`, `u32` e `u64`, permite separar dimensões arquiteturais e criptográficas do custo de privacidade.

As principais correções metodológicas incorporadas são: explicitação do vínculo entre valor privado e compromisso público, qualificação das garantias criptográficas herdadas do backend, distinção entre linha de base `Field` fraca e linha de base `Field` semanticamente controlada, fixação operacional da stack, antecipação do risco de deploy EVM sob EIP-170, detalhamento do benchmark WASM e separação entre *execution gas*, *calldata gas* e projeções com blobs.

No TCC II, a continuidade natural deste projeto será a implementação efetiva dos circuitos, a geração dos verificadores, a execução dos benchmarks, a coleta das métricas e a análise crítica dos resultados. Dessa forma, mesmo antes da implementação, o presente documento define um caminho de pesquisa reproduzível, com hipóteses, métricas e critérios claros para avaliar se a solução proposta é tecnicamente viável no cenário selecionado.

A Mapeamento técnico preliminar dos circuitos em Noir

A.1 Circuito conceitual de intervalo puro

A variante mais simples do circuito prova apenas que v pertence ao intervalo $[a, b]$. Ela é útil como baseline conceitual, mas não deve ser usada isoladamente para sustentar validação de atributo verificável.

Listing 1: Esboço conceitual de Range Proof puro em Noir.

```
fn main(v: u64, a: pub u64, b: pub u64) {
    assert(a <= b);
    assert(v >= a);
    assert(v <= b);
}
```

A.2 Circuito conceitual com compromisso público

A variante principal adiciona um compromisso público C . O esboço abaixo é conceitual: a função de hash ou compromisso exata será aquela disponível e estável na stack fixada.

Listing 2: Esboço conceitual de Range Proof com compromisso público.

```
fn main(
    v: u64,
    salt: Field,
```



```

    a: pub u64,
    b: pub u64,
    subject_id: pub Field,
    commitment: pub Field
) {
    assert(a <= b);
    assert(v >= a);
    assert(v <= b);

    // Funcao conceitual. Na implementacao final, usar primitiva
    // ZK-friendly disponivel na versao fixada do Noir/Barretenberg.
    let computed_commitment = commit(v as Field, salt, subject_id);
    assert(computed_commitment == commitment);
}

```

A.3 Variante Field com restrição explícita de bits

A comparação justa com tipos inteiros exige que o domínio de `Field` seja limitado explicitamente. O pseudocódigo abaixo expressa a ideia geral.

Listing 3: Esboço conceitual de `Field` com domínio controlado.

```

fn main(v: Field, a: pub Field, b: pub Field) {
    v.assert_max_bit_size::<32>();
    a.assert_max_bit_size::<32>();
    b.assert_max_bit_size::<32>();

    assert(a <= b);
    assert(v >= a);
    assert(v <= b);
}

```

A.4 Testes negativos mínimos

A bateria mínima de testes negativos deverá incluir:

1. $v = a - 1$ deve falhar.
2. $v = b + 1$ deve falhar.
3. $a > b$ deve falhar.

4. C incompatível com (v, r, id) deve falhar.
5. id incorreto deve falhar.
6. Reuso de proof contra verifier de circuito incompatível deve falhar.

Referências

- [1] GOLDWASSER, Shafi; MICALI, Silvio; RACKOFF, Charles. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, v. 18, n. 1, p. 186–208, 1989.
- [2] FIAT, Amos; SHAMIR, Adi. How to prove yourself: practical solutions to identification and signature problems. In: *Advances in Cryptology – CRYPTO '86*. Berlin: Springer, 1987. p. 186–194.
- [3] PEDERSEN, Torben Pryds. Non-interactive and information-theoretic secure verifiable secret sharing. In: *Advances in Cryptology – CRYPTO '91*. Berlin: Springer, 1992. p. 129–140.
- [4] BÜNZ, Benedikt et al. Bulletproofs: short proofs for confidential transactions and more. In: *IEEE Symposium on Security and Privacy*. 2018. p. 315–334.
- [5] GABIZON, Ariel; WILLIAMSON, Zachary J.; CIOBOTARU, Oana. PLONK: permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive, Report 2019/953, 2019.
- [6] BENARROCH, Daniel et al. A benchmarking framework for (zero-knowledge) proof systems. ZKProof Workshop, 2020. Disponível em: <https://docs.zkproof.org/pages/standards/accepted-workshop3/proposal-benchmarking.pdf>.
- [7] ERNSTBERGER, Jens et al. zk-Bench: performance benchmarking of SNARKs. Disponível em: <https://github.com/zkbench/zkbench>. Acesso em: maio 2026.
- [8] NOIR LANG. Noir Documentation. Disponível em: <https://noir-lang.org/docs/>. Acesso em: jun. 2026.
- [9] NOIR LANG. Fields. Noir Documentation. Disponível em: https://noir-lang.org/docs/dev/noir/concepts/data_types/fields. Acesso em: jun. 2026.
- [10] NOIR LANG. Integers. Noir Documentation. Disponível em: https://noir-lang.org/docs/dev/noir/concepts/data_types/integers. Acesso em: jun. 2026.
- [11] AZTEC LABS. Barretenberg Documentation. Disponível em: https://barretenberg.aztec.network/docs/getting_started/. Acesso em: jun. 2026.

- [12] AZTEC LABS. Aztec’s core cryptography, now in Noir. Aztec Network Blog, 2026.
Disponível em:
<https://aztec.network/blog/aztecs-core-cryptography-now-in-noir>.
Acesso em: maio 2026.
- [13] BUTERIN, Vitalik. Ethereum: a next-generation smart contract and decentralized application platform. 2013. Disponível em: <https://ethereum.org/whitepaper/>.
Acesso em: maio 2026.
- [14] WOOD, Gavin. Ethereum: a secure decentralised generalised transaction ledger. Ethereum Yellow Paper. Disponível em:
<https://ethereum.github.io/yellowpaper/paper.pdf>. Acesso em: maio 2026.
- [15] BUTERIN, Vitalik. EIP-170: Contract code size limit. Ethereum Improvement Proposals, 2016. Disponível em: <https://eips.ethereum.org/EIPS/eip-170>.
Acesso em: jun. 2026.
- [16] BUTERIN, Vitalik et al. EIP-4844: Shard blob transactions. Ethereum Improvement Proposals, 2022. Disponível em: <https://eips.ethereum.org/EIPS/eip-4844>.
Acesso em: jun. 2026.
- [17] WAHRSTÄTTER, Toni; BUTERIN, Vitalik. EIP-7623: Increase calldata cost. Ethereum Improvement Proposals, 2024. Disponível em:
<https://eips.ethereum.org/EIPS/eip-7623>. Acesso em: jun. 2026.
- [18] W3C. WebAssembly Core Specification. Disponível em:
<https://webassembly.github.io/spec/core/>. Acesso em: maio 2026.
- [19] V8 TEAM. WebAssembly compilation pipeline. Disponível em:
<https://v8.dev/docs/wasm-compilation-pipeline>. Acesso em: maio 2026.
- [20] GOOGLE. Using WebAssembly threads in C/C++ and Rust. web.dev, 2021.
Disponível em: <https://web.dev/articles/webassembly-threads>. Acesso em:
maio 2026.
- [21] BRUNNER, Edgar; MUNZEL, Ullrich. The nonparametric Behrens-Fisher problem: asymptotic theory and a small-sample approximation. *Biometrical Journal*, v. 42, n. 1, p. 17–25, 2000.
- [22] HOLM, Sture. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, v. 6, n. 2, p. 65–70, 1979.
- [23] NOETHER, Shen; MACKENZIE, Adam; others. Ring confidential transactions. *Ledger*, v. 1, p. 1–18, 2016.

- [24] SZABO, Nick. Smart contracts. 1994. Disponível em:
<https://www.fon.hum.uva.nl/rob/Courses/InformationInSpeech/CDROM/Literature/LOTwinterschool2006/szabo.best.vwh.net/smart.contracts.html>.
Acesso em: maio 2026.